

Pattern Ladder

A search engine for people who cannot name what they are looking for

This document explains a working system from first principles. It assumes no background in information retrieval, machine learning, or graph theory. It covers what the system does, every concept needed to understand how, every significant decision and the reasoning behind it, and — at length — the decisions that turned out to be wrong.

That last section is the most useful part. A system described only by its final state teaches you what somebody chose; a system described with its wrong turns teaches you how to choose.

Part 1 — The problem

What a student actually types

A student practising algorithm problems gets stuck in a specific way. They fail a problem, look at the solution, understand it, and still cannot say what *category* of thing they just failed. The next time they meet the same technique wearing different clothes, they fail again.

What they can describe is the mechanic:

"I keep failing problems where you shrink a window from the left."

That sentence contains no technical term. The technique it describes is called the *sliding window*, and the phrase "sliding window" does not appear anywhere in it. Nor do the words that appear in the problems themselves — those say things like "return the minimum length subarray whose sum is at least target".

This is the exact query a conventional search engine cannot serve, because conventional search finds documents containing your words, and the student's words are not in any document.

What the system returns

Three things, in place of a list of links.

The pattern. A named family of problems that all drill the same idea, so the student learns a category rather than a solution.

A ladder. Those problems in an order you can actually walk: easiest first, hardest last, so each one is reachable from the last.

A reason. For every recommendation, a statement of why it is there — which family it belongs to, how hard it is, how often people solve it, and whether it was found by matching your words or your meaning.

The idea the whole thing rests on

Every LeetCode problem page carries a "similar questions" list, curated by whoever maintains the problem. Two Sum links to 3Sum; 3Sum links to 4Sum; Minimum Window Substring links to Longest Substring Without Repeating Characters.

Individually those are trivia. Collectively they are a **graph** — a network of 2,830 problems joined by 1,932 human-drawn edges. And a graph can be *clustered*: you can ask which groups of problems are more connected to each other than to everything else. Those groups turn out to be recognisable techniques, and crucially they are **not** the same as LeetCode's own topic tags. The tags say "Array, Hash Table". The graph says "these eleven problems are all the same trick".

Sorting one of those groups by difficulty turns a search result into a study plan. That is the product.

Part 2 — Every concept you need, from scratch

Read this part if terms like BM25, embedding, cross-encoder or modularity are unfamiliar. Skip to Part 3 if they are not.

2.1 What a search engine is doing

You have a **corpus**: a fixed collection of documents. Here, 2,830 problem statements. You get a **query**: a piece of text from a user. Your job is to put the documents in order, most useful first.

Everything else is detail about how "useful" gets estimated.

Two properties matter, and they pull against each other:

- **Recall** — of all the documents that *should* be returned, what fraction did you return? A system with perfect recall never misses anything.
- **Precision** — of the documents you returned, what fraction *should* have been? A system with perfect precision never returns junk.

You can trivially get perfect recall by returning the entire corpus, and often get high precision by returning only the single most obvious result. Neither is useful. Real systems trade between them, and much of this report is about where that trade was made and why.

2.2 Lexical retrieval, and BM25

The oldest workable idea: **score a document by how often the query's words appear in it.**

Refine it three times and you have BM25, which has been the strong baseline in this field for thirty years.

Refinement one — rare words matter more. If a query contains "the" and "monotonic", a document containing "monotonic" tells you far more. This is *inverse document frequency*: a word appearing in few documents carries more weight. It is why "the" is nearly ignored for free.

Refinement two — repetition saturates. A document mentioning "stack" twenty times is not twenty times more about stacks than one mentioning it once. BM25 applies diminishing returns, controlled by a parameter called k_1 .

Refinement three — length is normalised. A long document contains more of every word by accident. BM25 discounts matches in long documents, controlled by a parameter called b .

BM25's strength is that it is exact, fast, needs no training, and cannot hallucinate a match: if it scores a document above zero, the query's words are genuinely in it.

Its fatal weakness for us: it scores our student's query near zero against every sliding-window problem, because they share no words.

2.3 Embeddings, and dense retrieval

The complementary idea: **represent meaning as a position in space.**

A neural network is trained to convert a piece of text into a list of numbers — here, 384 of them — such that texts with similar meanings get similar lists. That list is called an **embedding** or a **vector**, and you can picture it as coordinates in a 384-dimensional space, where nearby points mean similar things.

The comparison is **cosine similarity**: the angle between two vectors. Same direction means the same meaning, regardless of length. If every vector is scaled to length 1 in advance — *normalised* — cosine similarity becomes a plain dot product, which is a single multiply-and-add per dimension, and a whole corpus can be scored in one matrix multiplication.

This is what handles our student. "Shrink a window from the left" and "return the minimum length subarray whose sum is at least target" share no words, but the model was trained on enough text that it puts them near each other anyway.

Dense retrieval has a corresponding weakness: it has no notion of exactness. Ask for "Dijkstra" and it will cheerfully return shortest-path problems that never mention Dijkstra, because they are *about the same area*. And it always returns something — its nearest neighbours exist however far away they are — so a nonsense query gets confident nonsense back.

2.4 Why you use both, and how you combine them

The two methods fail on opposite queries. BM25 fails on paraphrase; dense fails on exact jargon. Running both and merging is called **hybrid retrieval**, and it is the default in current practice for exactly this reason.

The difficulty is that their scores are incomparable. BM25 produces unbounded positive numbers whose scale depends on the corpus and the query. Cosine similarity produces numbers between -1 and 1. There is no principled conversion, and normalising each to a 0-1 range per query is unstable — if one retriever returns a single result, that result gets a normalised score of 1.0 by construction.

Reciprocal Rank Fusion sidesteps the problem by throwing the scores away and using only the *positions*:

```
score(document) = sum over retrievers of 1 / (k + rank)
```

A document ranked first by one retriever and third by another scores $1/(60+1) + 1/(60+3)$. The constant k (conventionally 60) damps the advantage of the very top ranks, so one confident-but-wrong retriever cannot dominate.

This genuinely discards information — RRF cannot tell a runaway top hit from a marginal one. It buys robustness in exchange, which is the right trade when the two inputs disagree by design.

2.5 Bi-encoders and cross-encoders

Everything above is a **bi-encoder** arrangement: the query is turned into a vector, each document was turned into a vector earlier, and they are compared. The document never sees the query. That is what makes it fast — the documents are embedded once, offline, and a search is one matrix multiply.

A **cross-encoder** does the opposite. It takes the query and one document *together* as a single input and reads them jointly, so it can represent relationships between specific words in the query and specific words in the document. It can tell "this document answers this question"

from "these two texts are about similar things".

The cost is brutal: it is one neural network forward pass **per document**. You cannot run it over 2,830 documents per query. So it is used as a **reranker** — retrieval produces a shortlist of 50 candidates cheaply, and the cross-encoder reorders just those.

Note what this implies, because it matters later: **a reranker can only reorder what retrieval gave it**. If the right answer is not in the 50, no amount of reranking finds it.

2.6 Graphs, communities, and Louvain

A **graph** is dots (**nodes**) joined by lines (**edges**). Here each problem is a node, and an edge means "these two are listed as similar".

Some vocabulary that appears throughout:

- **Degree** — how many edges a node has.
- **Isolated node** — degree zero. Nothing links to it.
- **Connected component** — a group where you can walk from any node to any

other. A graph in many components is fragmented.

- **Weight** — a number on an edge saying how strong the connection is.

Community detection asks: which groups of nodes are more densely connected to each other than to the rest? The standard answer is the **Louvain method**, which optimises a quantity called **modularity** — how much more internal linkage each proposed grouping has than you would expect from a random graph with the same degrees.

Two properties of Louvain matter a great deal in practice:

It is stochastic. It visits nodes in random order and gives a different answer each run. Without a fixed random seed, your problem families are renumbered and reshuffled on every rebuild.

It has a resolution parameter. Above 1.0 it prefers many small communities; below, few large ones. This single number decides whether you get "one family per technique" or "one family per broad topic", and there is no correct value — it depends entirely on what the communities are for.

There is also a trap worth knowing. **Modularity is trivially high on a fragmented graph**. If your graph is in a thousand disconnected pieces, any clustering scores near 1.0, because there are no edges between the pieces to violate. A high modularity is therefore not evidence of good clustering unless you also know the graph is connected. This project's raw graph scores 0.93, and that number means almost nothing.

2.7 What a language model contributes

One narrow job: turning a sentence into a structure.

```
"I keep failing problems where you shrink a window from the left"  
  v  
{ "technique": "sliding window", "difficulty": null, "mode": "ramp" }
```

That recovered phrase, "sliding window", is jargon the student did not write and the retrievers badly need — it is precisely what BM25 can match on.

Two engineering properties dominate this step:

Models return text, not data. A model asked for JSON may return JSON wrapped in a code fence, JSON after a sentence of preamble, or an apology. Every one of those must be handled, because the alternative is a crash on a live query.

A model is a remote dependency. It has a rate limit, a network in front of it, and a bad day. If your system stops working when it does, you have built a system that stops working.

2.8 Measuring any of this

You cannot improve what you cannot measure, and the measurements used here are:

- **hit@k** — for a set of queries with known correct answers, how often is the correct answer in the top k?
- **Coverage** — what fraction of the corpus is in some family at all? A problem in no family cannot produce a ladder.
- **NMI (Normalised Mutual Information)** — how much do two groupings of the same items agree? 1.0 means identical; 0.0 means unrelated. Used here to test whether the discovered families are just LeetCode's tags in disguise.
- **Coherence** — what fraction of a family's members actually carry one of the tags the family is named after? A low number means the name is a false promise.

Part 3 — What was measured before writing any code

The plan this project started from made several factual claims. Checking them first changed the design, and cost about twenty minutes.

3.1 The dataset was the wrong file

The plan named a Hugging Face dataset with "2,823 problems with full descriptions, tags, difficulty, acceptance rate, hints and the similar-questions graph".

The dataset is real and MIT-licensed. But its **default configuration** — what you get by loading it the obvious way — has exactly two columns, `user_queries` and `expected_output`. It is an instruction-tuning set. It has 2,823 rows, which is where the plan's number came from, and it contains none of the fields the project needs.

The structured table exists only under `raw_data/leetcode_problems.json`, with **3,549 rows** and 22 fields.

	count
Rows in the structured file	3,549
Paid-only, statements empty	-716
Empty after HTML stripping	-3
Retrievable corpus	2,830

The headline number was right by coincidence — 2,823 is the row count of a different file that happens to land within seven of the truth.

3.2 The graph is sparse — measured, not assumed

The plan said the graph was load-bearing and its density could not be known until the first day of building. It can be known in ten minutes.

	measured
Nodes / edges	2,830 / 1,932
Problems with no link	1,074 (38.0%)
Mean degree among the connected	2.2 (median 2, max 19)
Connected components	1,248 (largest holds 1,221)
Louvain families of ≥ 5	60
Corpus inside a family	49.2%

The plan's "40+ families" claim survives. But **half of all searches would return no ladder**, and the ladder is the entire product.

3.3 The proposed fallback was worse than the problem

The plan's contingency: if the graph is too sparse, build it from problems sharing two or more tags instead.

Tested. It produces 9 families of ≥ 5 members, sized 303, 290, 184, 180, 110, 107, 104, 50, 5. They are enormous, and their members share exactly the tags they were joined on — it reconstructs the tag taxonomy that the project's central claim is about transcending.

So the contingency would have preserved the schedule and destroyed the point. Part 5 covers what replaced it.

3.4 The free tier is 14x smaller than stated

The plan said Groq's free tier gives 30 requests a minute and 14,400 a day.

Those are the limits for llama-prompt-guard-2-22m and -86m, which are classifiers and cannot emit structured JSON. Every chat model that can do the job runs on **30 RPM / 1,000 RPD / 8,000 tokens per minute / 200,000 tokens per day**.

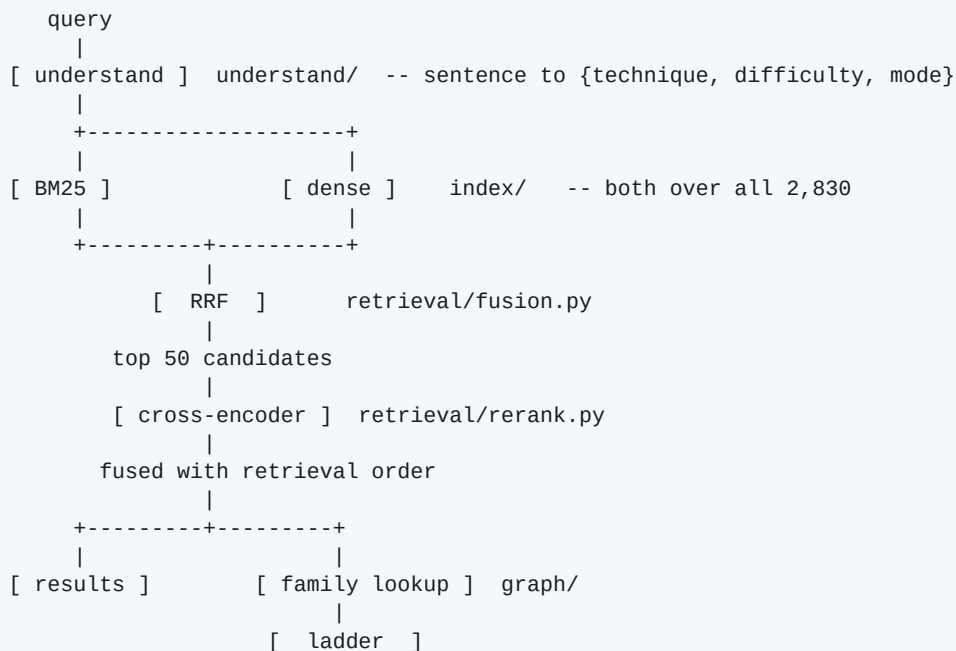
The binding constraint is tokens per minute, not requests per day. The plan called for two model calls per query — one to parse, one to write explanations — at roughly 1,200 tokens, which caps throughput at about **six queries a minute**. That drove a design change described in Part 5.

3.5 The deployment target is no longer free

The plan targeted Hugging Face Spaces' free CPU tier. From Hugging Face's own documentation, current: Gradio and Docker Spaces require a paid plan for personal accounts, and Streamlit is no longer offered as an SDK at all.

The free replacement is Streamlit Community Cloud, which imposes roughly a 1 GB memory ceiling per app — a real design pressure, because the two models are tiny but PyTorch is not.

Part 4 — The architecture, module by module



config.py — every tuned number, with its evidence

One module holds every constant in the system, and each carries a comment saying what was measured to choose it. This is not tidiness; it is the difference between a parameter and a guess. A reader who wants to know why the Louvain resolution is 8.0 finds the answer at the definition, along with what went wrong at 2.8.

data.py — what counts as a problem

Downloads the corpus, filters it, normalises it. Two exclusions carry real weight:

Paid-only problems are dropped. Their statements are empty. Keeping them would put ~700 textless documents in the index. They could never be retrieved — but they *would* still change every other document's score, because BM25 normalises by average document length, and 700 empty documents drag that average down.

Links pointing outside the corpus are dropped. A retained link to an excluded problem creates a graph node with no text, which surfaces in a ladder as an entry the student cannot open.

Ordering is by problem id, always. This looks cosmetic and is not: the embedding matrix and the BM25 index are **positional** — row 7 is problem 7 — so a corpus that came back in a different order after a rebuild would silently misalign every result, with nothing raising an error.

text.py — HTML to plain text

LeetCode statements are HTML fragments. Converting them naively causes two failures, both silent:

Welding. `re.sub(r'<[>]+>', ' ', "<p>a</p><p>b</p>")` gives `"ab"` — a token matching nothing. The fix is to emit a word boundary at every block-level tag.

Non-breaking spaces. These statements are full of `nums ` and `numsand`. Decoded but not normalised, `U+00A0` is not treated as whitespace by every tokeniser, and `numsand` becomes an unmatchable token.

Neither raises. Both just quietly cost recall, which is why this is a deliberate parser rather than a regex.

This module also composes the string that gets indexed: the title repeated three times, then the tags, then the statement. BM25 has no concept of fields, so repeating the title is the only way to say a title match is worth more than a body match. Three is not a guess — see Part 7.

`index/lexical.py` — **BM25**

A wrapper over `bm25s` whose real job is guarding tokenisation consistency. A query tokenised with different settings than the corpus retrieves nothing useful, and does not error. Queries are tokenised to raw strings and mapped through the *index*'s vocabulary, so the query cannot carry its own vocabulary that merely happens to agree.

Documents scoring zero are dropped rather than returned, and a query that is entirely stopwords returns nothing at all — letting the dense arm carry it is better than surfacing arbitrary documents.

`index/dense.py` — **embeddings**

Embeds documents with `all-MiniLM-L6-v2`, normalised at encode time so every later comparison is a dot product and cosine similarity cannot drift apart from inner product in different call sites.

Search is exhaustive: 2,830 x 384 floats is 4.3 MB and a full scan is a single BLAS call at about a millisecond. An approximate index (FAISS, HNSW) would add a dependency, a build step and an approximation error to save nothing measurable at this scale.

The encoder is **injectable**. This was not the original design; it was forced by testing, and Part 6 explains why that was the right pressure to yield to.

`retrieval/fusion.py` — **RRF**

Twenty lines. The only subtlety is that ties break on document id, so that two documents with identical fused scores cannot swap between runs and make the evaluation flaky.

`retrieval/rerank.py` — **the cross-encoder**

Scores query-document pairs. Its outputs are raw logits: unbounded, usually negative, and meaningful only relative to each other within one query. They are deliberately not squashed into `[0,1]`, because a bounded number invites the interface to present it as a confidence, which it is not.

`retrieval/search.py` — the pipeline

The ordering here is load-bearing:

- Both retrievers run over the **whole** corpus. Running one over the other's output would inherit the first one's misses, which defeats the purpose of having two.
- Fusion happens **before** reranking, so the cross-encoder sees candidates that neither arm alone ranked highly.
- The cross-encoder runs on 50 documents, not 2,830.
- Family expansion happens **last**, on survivors only.

Difficulty filtering is applied to the fused pool rather than to the final results, because filtering afterwards routinely leaves fewer than `k` results. And if a requested difficulty band is empty, it is ignored with a note rather than returning nothing — an over-specific parse should not silently erase every result.

`graph/build.py` — the graph and its backfill

Builds the curated link graph, then attaches under-connected problems to embedding neighbours under three simultaneous constraints: the node must have almost no curated links, the neighbour must clear a similarity floor, and the two must share a topic tag. Inferred edges carry a lower weight than curated ones, so Louvain — which optimises *weighted* modularity — still lets the human links decide where boundaries fall.

`graph/families.py` — communities, names, ladders

Runs Louvain seeded and sorted. Names each family by the tags most over-represented in it relative to the corpus (ranking by raw count would name every family "Array"), plus the most distinctive word from its members' titles, which supplies the concrete noun a tag list misses — "Palindrome", "Subarray".

Also holds the evaluation metrics, deliberately: a metric that lives next to the thing it measures is harder to quietly stop running.

`understand/` — query understanding, and its safety net

`schema.py` validates each field independently, so a nonsense difficulty does not discard a good technique. `fallback.py` is a deterministic keyword parser covering the phrasings students use when they cannot name a technique. `groq_client.py` makes the API call, and its contract is absolute: **it always returns an Intent**. Network failure, missing key, rate limit, malformed JSON, prose instead of data — all degrade to the fallback.

The fallback matters more than the prompt. It is good enough to run the entire system unaided, which is what makes the project testable and demonstrable with no key at all. Every number in this report was produced with it.

`explain.py` — grounded, not generated

Every explanation is assembled from fields that were actually retrieved. No model writes them.
Part 5 explains why this reverses the original design.

Part 5 — The decisions, and the reasoning

5.1 Fixing the sparse graph without rebuilding the tags

The problem. 38% of problems have no curated link; clustering the raw graph leaves 49.2% of the corpus with no family, so half of all searches return no ladder.

The rejected fix. Tag co-occurrence, as the original plan proposed. Measured: 9 giant families whose members share exactly the tags they were joined on. It solves coverage by destroying the claim.

The chosen fix. Attach only under-connected problems, only to embedding neighbours above a similarity floor, only when they share a tag, and only at a reduced edge weight.

Each constraint has a job. Restricting to under-connected nodes stops inferred edges bridging and merging established families. The similarity floor stops weak attachments. The shared-tag test — a *conjunctive* constraint, added after pure similarity was found to place "Valid Anagram" in a shortest-path family — blocks textually similar but topically unrelated pairs. The reduced weight keeps curated links deciding boundaries.

It is worth being clear that using tags as a *filter* is not the same as using them as a *source*. The filter only removes candidate edges that embedding similarity already proposed; it can never create one. The guard on that distinction is the tag-agreement metric, reported on every build.

Result: coverage 49.2% -> 86.3%, families 60 -> 137, coherence 0.790 -> 0.845, tag agreement 0.618 -> 0.667.

5.2 The reranker is fused, not obeyed

The original plan called the cross-encoder "the largest single quality jump available". Measured on the evaluation set:

configuration	hit@5	oblique queries	mean rank of hits	latency
No reranking	0.90	1.00	1.56	0.016 s
Cross-encoder decides	0.85	0.83	1.06	1.44 s
Fused with RRF	0.95	1.00	1.53	1.32 s

Letting the cross-encoder overwrite the retrieval order *lowers* hit@5, and lowers it most on the obliquely-phrased queries this project exists to serve. It does sharpen precision considerably. So it is a trade, not an upgrade — and fusing the two rankings beats both.

5.3 Explanations are assembled, not generated

The original design called a language model a second time to write a justification per result. Two reasons that was reversed.

Budget. On the free tier the binding limit is 8,000 tokens per minute. A second call roughly doubles token spend for the least load-bearing part of the answer.

Honesty, which matters more. A model asked to justify a ranking it did not produce will write a plausible reason whether or not a real one exists. A confident false explanation attached to a *correct* result is worse than no explanation, because the student has no way to detect it and learns a relationship that is not there.

So explanations state only what the pipeline established: which retrieval arm found it, which family it is in, its difficulty and acceptance rate, and — only when the two genuinely share a family — how it relates to the top result.

5.4 The system shows its own reading

The parsed intent is displayed before any results, labelled with whether it came from the model or the offline parser. A student who cannot name a technique cannot tell a good result from a bad one, so the system's interpretation must be inspectable. A tool that is silently wrong is trusted until the first time it is silently wrong, and then not at all.

5.5 Determinism as a requirement

Louvain is seeded. Corpus order is fixed. Every tie in every sort has a final deterministic key. Nothing reads the clock. The model call uses temperature 0.

This is not fastidiousness. Positional artefacts (embedding rows, BM25 doc ids) misalign silently; unstable ladders make the evaluation flaky; unseeded Louvain rennumbers every family on every rebuild, so no result is reproducible and no regression is detectable.

Part 6 — What was wrong, and how it was found

This is the important part.

6.1 The kNN backfill drowned the signal it was meant to help

What was built. Attach under-connected problems (degree ≤ 1) to their 4 nearest embedding neighbours above 0.55 similarity.

What happened. 5,494 inferred edges added to 1,932 curated ones — a 3:1 ratio. Coverage rose from 49.2% to 97.3%, which looked like a triumph. But family count *fell* from 54 to 27, and the largest family reached **260 problems**.

Why it was wrong. A 260-member family is a topic, not a pattern, and its ladder is not a study plan. Worse, at 3:1 the curated links had stopped deciding anything; the clustering had become embedding clusters wearing a similar-questions costume. The docstring of the function that did this said, in so many words, that this must not happen — and then it did, because nothing measured it.

The fix. A parameter sweep with a stated objective: maximise the share of the corpus in a *walkable* family, subject to keeping enough families to be useful. Neighbours 4 $\rightarrow$ 1, similarity floor 0.55 $\rightarrow$ 0.65, and later a shared-tag requirement.

The lesson. Coverage alone is not a goal. Any coverage target can be met by connecting everything to everything, and the resulting single family is worthless.

6.2 A metric that measured its own blind spot

To test "these families are not LeetCode's tags", NMI was computed between the family grouping and a tag-derived grouping across the whole corpus.

It reported 0.75 for the raw link graph — high enough to suggest the families *were* largely tags, which would have falsified the project's central claim.

It was wrong, and wrong in a specific, instructive way. Roughly half the corpus lands in no family, and those problems each received a *unique* label in the family grouping. Many also carry a rare tag combination, giving them a near-unique label in the tag grouping too. The two groupings therefore "agreed" on an enormous block of mutual singletons, and NMI was dominated by that agreement — by the problems that had been left out — rather than by anything about the families.

Restricting the computation to problems that actually received a family dropped the figure from 0.75 to 0.60.

The lesson. A metric can be confidently, quietly wrong. This one would have caused the project's main claim to be abandoned on the strength of an artefact. The tell was that the number did not move when the clustering changed a lot.

6.3 A fix for a defect that did not exist

Inspecting the families by eye showed real problems: "Valid Anagram" in a heap/shortest-path family, "Find Center of Star Graph" in a binary-tree family. Those strays were Easy with high acceptance rates, so they sorted to the *top* of their ladders — the most visible part of the output was the least reliable.

A metric was built for it — coherence at the head of the ladder — and an elegant fix followed: partition each family into "core" members (with curated links) and "peripheral" ones (attached only by inference), and build ladders from the core.

Head coherence went down. 0.753 to 0.732.

The diagnosis had been wrong. Measuring properly showed the ladder head is 84% Easy problems, and Easy problems carry **2.54 tags on average against 3.90 for Hard**. Fewer tags means fewer chances to intersect the family's naming tags, so the metric was substantially measuring *tag count*, not belonging.

A controlled comparison settled it — same families, same metric, split by how each member got there:

provenance	n	mean fit to family profile
curated link	1,538	0.606
inferred edge	192	0.649

Inferred members fit their families *better* than curated ones. The bad examples had come from the earlier, looser configuration; the tightened one had already fixed them.

So the fix was deleted. It cost 4.4 percentage points of coverage to solve a problem that measurement said did not exist.

The lesson. An anecdote justifies an investigation, not a change. And when a fix does not move the number it was built to move, the honest response is to suspect the diagnosis rather than to tune the fix.

6.4 Optimising a proxy instead of the goal

The graph parameters were first swept against coverage and family size, because those were the only things measurable before an evaluation set existed. That sweep chose a Louvain resolution of 2.8.

Once the evaluation set could score whether the ladder came from a family that matched the question, 2.8 turned out to be badly under-split. A single 34-member community contained three unrelated patterns at once:

- queue and stack *design* problems (Implement Queue using Stacks, Min Stack)
- heap and greedy problems (Take Gifts From the Richest Pile)
- anagram and sliding-window problems (Group Anagrams, Minimum Window Substring)

So Minimum Window Substring lived in a family named "**Queue / Design**", and the query the entire project was designed around — "shrink a window from the left" — returned a ladder of queue problems, including *Valid Anagram* and *Group Anagrams*.

Re-sweeping against ladder quality moved the resolution from 2.8 to 8.0:

	resolution 2.8	resolution 8.0
Families	86	137
Largest family	60	42
Ladder from the expected family	0.583	0.833
Coverage	85.1%	86.3%
Name coherence	0.828	0.845

The lesson. Optimising a proxy gets you a good proxy score. The sweep was methodologically sound and the answer was still wrong, because the objective was a stand-in. Build the evaluation set earlier than feels necessary.

6.5 The ladder was built from one noisy sample

Family selection originally anchored on the top-ranked result's family, with a comment explicitly arguing against consensus: "a single strong hit is a better anchor than a majority vote among weaker ones".

That reasoning was wrong. One result's family is one noisy sample of what a query is about. When *Sliding Window Maximum* ranked first and its family was "Queue / Design", the entire ladder followed it off a cliff.

Replaced with rank-weighted consensus across the results plus a bonus when the parsed technique appears in a family's own name or tags — which is only possible because the query was parsed into a technique in the first place. The query-understanding stage pays for itself twice.

A third signal was tried and rejected: scoring families by the mean similarity of their best members to the query. It sounds obviously helpful. It made things worse at every weight from 1 to 8 (0.583 -> 0.500), because families are broad enough that their best few members resemble almost any query in the neighbourhood. That negative result is recorded in the code, where someone would otherwise re-add it.

6.6 The right family, and a useless ladder

With family selection fixed, a new failure appeared. "Detect whether a linked list has a cycle" retrieved *Linked List Cycle*, *Linked List Cycle II* and *Intersection of Two Linked Lists* as the top three — correct — and served a ladder consisting **entirely of digit-arithmetic problems**.

The family was right. The three correct problems were in it. But the ladder sorted by difficulty then acceptance rate, and the family's easiest, most-accepted members were unrelated Easy problems.

This is why `ladder_hit` exists as a metric, independent of family naming: it asks whether the canonical problem for the query is on the ladder at all. It scored **0.40**. The ladder — the actual product — was the weakest part of the system, and the two metrics that existed both looked healthy.

The fix. Relevance selects which members are eligible; difficulty then orders the survivors. A member must be within 80% of the best member's similarity to the query. Doing it the other

way round — sorting by relevance overall — produces a good list that is not a ladder, because a ladder's value is that it ascends.

`ladder_hit`: **0.40 -> 0.85**, with no change to `hit@5` or family accuracy.

One more correction followed immediately: a strict floor sometimes left a single rung. A one-rung ladder is a search result wearing a ladder's label, so a minimum of five is enforced by relaxing the filter when necessary.

6.7 A comment that claimed a measurement that never happened

The function composing indexed text carried this comment:

"Three is not a guess -- it is the value that won the sweep in eval/README, and the sweep is reproducible via `scripts/sweep_title_weight.py`."

Neither the file nor the sweep existed. The number was a guess wearing the costume of a measurement, which is worse than an obvious guess — a reader would have trusted it.

The sweep was then actually written and run:

title repeats	pool recall
1	0.950
2	0.988
3	1.000
5	1.000

Three is correct, and is now the smallest value that reaches full pool recall. The guess was right; the comment was still a lie until the measurement existed.

6.8 Untestable by construction

The search pipeline called a module-level `encode()` function directly, hardwiring it to the real 384-dimensional model. Every test touching search therefore needed a 90 MB download and compared 384-dimensional query vectors against 3-dimensional fixture vectors.

The test suite was not failing because of a bad test. It was failing because the design made offline testing impossible. Making the encoder injectable fixed both the tests and a genuine coupling problem — the index now owns its own vector space instead of assuming a global one.

The lesson. When tests are hard to write, that is usually information about the design rather than about the tests.

6.9 The ladder depended on a presentation setting

Found by running the actual interface rather than the evaluation harness.

The evaluation ran with five results; the interface defaults to ten. On the project's flagship query, ten results chose the correct sliding-window family and five did not — because family selection voted over *the results being displayed*, so moving a slider in the sidebar changed

which pattern the student was taught.

That is indefensible on its face. How many rows fit on screen is a presentation choice and must not reach into the recommendation. Family voting now reads a fixed depth regardless of what is shown.

The aggregate metrics barely moved, which is the point worth noting: this was not a scoring bug, it was a **coupling** bug, and no amount of running the evaluation harness would have surfaced it. It took using the thing.

6.10 Three defects that only a real API call could reveal

The model path had been written, reviewed, and covered by tests that injected every failure mode: missing key, network error, malformed JSON, prose instead of data. All of them passed. The first real call found three separate defects, and the reason none had surfaced is the same in each case: **they degrade silently to the fallback, and the fallback is good.**

The token budget failed every single call. `max_tokens` was 200. The configured model is a *reasoning* model: it emits several hundred tokens of internal reasoning before the answer, and those count against the same budget. Every request returned `400 json_validate_failed` with the object cut off mid-generation. The model path had a 0% success rate, and the system looked entirely healthy, because each failure fell through to the rule-based parser and produced a reasonable answer.

This is the most important thing in this report about testing. A well-built fallback is a liability for observability: it converts a total failure of the primary path into a slight quality regression that nobody notices. The offline suite could not have caught it, because the offline suite deliberately never calls the API.

The default reasoning effort was wasteful. Setting `reasoning_effort="low"` cost 295 tokens per query against 488 at the default and 558 at "high" -- and was also *faster* (0.64 s against 0.95 s) and *better*, extracting a technique the default returned null for. Better on every axis measured, which is unusual enough to be suspicious; it held across the full evaluation set.

The prompt made the model answer "single" to almost everything. The original wording offered the two modes symmetrically: "ramp" if they want a progression, "single" if they want one. On a hand-checked set the model got 2 of 5 right, and read "*I keep failing problems where you shrink a window from the left*" -- a description of repeated failure, and the clearest possible case for a progression -- as a request for one problem. Stating the default explicitly took it to 5 of 5, and incidentally improved technique extraction ("stack" became "monotonic stack"), for about 12% more tokens.

All three are now pinned by offline tests that assert the request parameters rather than making a request.

Part 6b — A second audit, component by component

Everything above was found while building. This section is what a deliberate pass over the finished system turned up afterwards, which is a different exercise: the code already worked, the tests already passed, and the numbers were already good.

Seven of the nine defects below produce no error and no visible symptom.

6b.1 A relative threshold that inverts on negative numbers

The ladder keeps family members scoring at least 80% of the best member's similarity to the query. For a positive best that is a sensible floor. For a negative one it is *above* the best -- 0.8 times -0.05 is -0.04 -- so the most relevant member fails its own threshold and the comparison starts selecting the *least* similar members.

Reachable: probing 1,507 query-family pairs found one with a negative best similarity. It never raised, because the minimum-rungs fallback quietly refilled the ladder; the only symptom was the relevance filter silently ceasing to work.

6b.2 An empty query returned five confident results

Searching for the empty string produced five ranked problems, per-result explanations, and a full ladder. Dense retrieval is exhaustive -- it embeds whatever it is given, however meaningless, and returns that point's nearest neighbours. So did a run of spaces, and so did "!!! ???".

The interface happened to guard against empty input, which is exactly why this survived: the engine is the public surface and was relying on its caller to be careful.

6b.3 Stale artefacts loaded without complaint

The embedding matrix and the BM25 index are positional -- row 7 is problem 7 -- and neither file records which corpus it was built from. Loading a corpus of 2,830 problems beside an embedding matrix of 100 rows succeeded silently and returned results; the first sign of trouble was an `IndexError` from inside a matrix slice, thousands of lines from the cause. Had the sizes matched but the *order* differed, there would have been no sign at all.

This is the failure mode Part 4 already named as the worst this system has, and nothing was checking for it. Three integers compared at load time now convert it into an error that names the problem and the fix.

6b.4 A name collision the tests could not see

Adding a parameter named `describe` to the build silently shadowed an imported function of the same name, so a call meant for that function became a call on the boolean `False`. The suite stayed green. Only the linter noticed.

The reason the suite could not catch it is the interesting part: the build function downloads a corpus and loads a model, so no offline test could reach it, and the whole graph-construction-and-clustering step therefore had **no test at all**. The fix is not the rename -- it is extracting the pure computation into its own function, which a test can now call with fixtures.

6b.5 Duplicate slugs, and problems similar to themselves

The loader did not enforce slug uniqueness. The slug keys problem lookup, family membership *and* the embedding row index, so two rows sharing one slug would not raise: the dictionaries keep whichever came last, one twin becomes unreachable, and the other is served under a row index belonging to its sibling.

The upstream file has no duplicates today. That is precisely why it needed enforcing rather than assuming -- nothing would have announced it if that changed. Self-referential similar-question links were being kept for the same reason, and would have let a problem be presented as preparation for itself.

6b.6 The container image could not have built

The dependency install ran before the source was copied, to keep that layer cacheable. It does not fail: it builds and installs an *empty* package. Copying the source afterwards does not repair that, because the install is a real one rather than editable, so the empty package keeps shadowing the real code and the import raises `ModuleNotFoundError` at the model warm-up step.

Worth recording that the first explanation of this was wrong. The obvious guess -- that the build backend would fail with no package to build -- was tested and is false; it builds an empty wheel quite happily. The real mechanism only appeared by running the sequence.

There was also no ignore file for the build context, so roughly 1.6 GB of virtualenv, model cache and indexes were being sent to the daemon on every build, all of which the image rebuilds for itself anyway.

Both the diagnosis and the fix were later confirmed by building the image for real. The warm-up step that would have raised `ModuleNotFoundError` under the old ordering -- importing the package to preload both models -- completed in 234 seconds, and a container run from the resulting image served a full query: parsed intent, ten ranked results, and a ladder with its curated-link provenance.

6b.7 The offline parser knew 8 techniques in 24

The fallback is the path that runs with no API key, and every number in the original report was produced with it. Tested against 24 ordinary queries it recovered a technique for 8.

Worse than the misses was one hit: "matrix spiral traversal" was read as a **tree** problem, because "traversal" was listed as a tree cue. The technique is appended to the query and steers retrieval, so a confidently wrong parse is more damaging than none at all.

Rewriting the vocabulary as phrases rather than bare words, and adding the techniques it simply lacked -- stack, queue, heap, matrix, sorting, hash table -- took it to 22 of 24, with no

confidently-wrong answers.

6b.8 A validation rule that rejected what it was meant to clean

Model-written family descriptions are normalised before being cached. Two ordering mistakes, in opposite directions:

- Whitespace was collapsed before the "did the model return a list?" check, so

the newline test could never fire.

- Punctuation was normalised before that same check, which turned every em-dash

into a hyphen-space -- and the list-marker rule then rejected it. **Every description containing an em-dash was silently discarded.**

A non-breaking hyphen also reached a cached description, which looks identical to a hyphen on screen and behaves differently everywhere else.

6b.9 Temperature 0 is not determinism

The code claimed, in a comment, that temperature 0 made the reading reproducible. Measured: repeating all twenty evaluation queries three times each, nineteen were bit-identical and one was not -- "detect whether a linked list has a cycle in it" came back as both "two pointers" and "tortoise and hare", both correct.

Temperature 0 is greedy sampling, not a reproducibility guarantee; served models batch and route in ways that break ties differently between calls. One observed run of the live evaluation scored 0.95 rather than 1.00 for this reason. The comment now says what was measured.

What this pass suggests

Seven of these nine produce no error. The two that eventually would -- the stale artefacts and the container build -- would do so far from their cause. The audit that found them was not clever: it was a set of deliberately stupid inputs (empty strings, mismatched files, duplicated rows) applied to each component in turn, plus a linter.

Part 6c — What was improved

Beyond the fixes, four changes that make the system better rather than merely correct.

The ladder now says why one rung follows another. The original brief's example promised "why 209 follows 3". That relationship is already on the record -- LeetCode's curated similar-questions list -- and was being used to build the graph but never surfaced to the student. A rung linked to an earlier rung now says so, and only when the link genuinely exists; there is no inferred fallback, because the honest answer to "why does this follow that" is sometimes nothing.

It also surfaces relationships a reader would not guess. "Linked List Cycle" shows as following "Happy Number", which is correct and non-obvious: both are solved by Floyd's cycle detection.

Result explanations cite the curated link too, in preference to the difficulty comparison. "Listed as similar to Two Sum" is true of nineteen problems; "a step up from Two Sum" is true of hundreds.

Families can be described rather than labelled. The tag-derived name is taxonomic -- and, for ten of the 137 families, not even unique: four separate families are all called "Tree / Binary Tree", which tells a student nothing about which of the four they are looking at. An optional offline pass writes a one-line description from each family's own member titles, turning "Monotonic Stack / Stack" into "monotonic stack to find next greater or visible elements". 120 of 137 families get one; the rest legitimately have no single shared technique, answer with the refusal token they were offered, and keep their name.

This is additive by construction. The deterministic name is untouched, every metric still scores against it, and a build with no key produces identical artefacts minus one optional field. Descriptions are re-validated on load as well as on generation, so a stored one can never be worse than the current rules.

The offline parser was rewritten, as above: 8 of 24 to 22 of 24.

Part 6d — A second enhancement pass

The audit in Part 6b asked "what is broken". This pass asked a different question of each component in turn: what does it already have that it is not using?

That framing found more than looking for bugs did, because the answer was concrete four times over. `hints`, `likes`, `dislikes` and `difficulty_spread` were all loaded from the dataset, carried through every layer, and read by nothing.

6d.1 Approval: a signal that was sitting in the corpus

Every problem carries a like and dislike count. The median problem has about 1,200 votes. Nothing used them.

The reason this matters is that the ladder was ordering rungs by acceptance rate, and acceptance rate answers a different question than it appears to. It says how often people who attempt a problem succeed. It says nothing about whether the problem was worth attempting. Measured across the corpus the two correlate at **+0.14** -- they are very nearly independent, so an ordering that knows only about acceptance is blind to an entire dimension.

Blind in a way that actively misleads, too. A badly-regarded problem is often badly regarded *for being guessable*, which inflates its acceptance rate and floats it to the top of its tier. The case that made this concrete:

***Design an Ordered Stream** -- 82% acceptance, **13% approval** across 4,115 votes. Under the old ordering it came first in its tier, and it was appearing at the head of a real ladder.*

Rungs are now ordered by a blend of the two. The like ratio is smoothed toward the corpus median with a prior worth 50 votes, so that a problem with three likes is treated as unremarkable rather than as excellent -- three likes is not evidence, it is an absence of evidence.

The result is a good example of why the evaluation set is not enough. On the twenty smoke queries the change moved mean ladder approval from 0.9034 to 0.9097 and nothing else at all -- a rounding error, easily dismissed. Measured across all 137 families instead:

	before	after
Family ladders whose top five changed	-	109 of 137
Poorly-regarded problems (approval < 0.60) in a ladder head	59	40

A third of the bad recommendations disappeared. The eval could not see it because twenty queries touch only a handful of families.

6d.2 A cache that fixes a correctness problem, not just a cost one

Part 6b.9 recorded that temperature 0 is not a determinism guarantee: one of twenty evaluation queries came back two different ways across identical calls. That is a real problem

for this system specifically, because it *shows the student its reading* -- and a reading that changes between identical questions is not something a student can check.

Caching readings on disk fixes it by construction. It also happens to remove 372 tokens and about 0.9 seconds from every repeat query, measured at a 42,000x speedup on the second call, but the reproducibility is the point.

The key covers the model and the **system prompt**, not just the query. That is what makes the cache safe to keep forever with no expiry: editing the prompt is the one change that should invalidate every stored reading, and it does so automatically by changing every key. Nothing else about a reading goes stale.

Two deliberate exclusions. Failed calls are not cached, because caching a fallback reading would make one bad minute permanent. And the offline parser is not cached, because it is already deterministic and free, so storing its output would add a staleness risk in exchange for nothing.

6d.3 Where a pattern leads

The graph is used to build families and then dropped. But curated links that *cross* a family boundary are also information: a person saying two problems are similar despite the clustering having separated them. Across the corpus there are 189 such family pairs, covering 111 of the 137 families.

A ladder answers "what should I practise next within this pattern". This answers the question that follows: which pattern comes after this one. Only curated links count -- two families being textually adjacent in embedding space is mostly evidence that both mention arrays.

Getting the ranking right took two corrections, both caught by looking at the numbers rather than the code:

The counts were all even. 171 pairs at 2, 14 at 4, 4 at 6, and nothing odd anywhere. That is not a distribution, it is a tell: the upstream lists are largely reciprocal, so counting link endpoints counts every relationship twice.

Raw counts favour large families. Two big families cross by coincidence more often than two small ones, so the strongest "next step" after monotonic stacks came out as *randomly picking elements uniformly from a set* -- purely because that family is large. Normalising by the square root of the two family sizes puts *Between / Array / Monotonic Stack* first, which is right.

6d.4 Hints, and the whole family at a glance

Hints ship with every problem and were being discarded. A study tool that holds hints and does not offer them is wasting the best thing it has.

They are shown behind a click, and only for the single problem the student is being told to start with. A hint is genuinely useful when someone is stuck and actively harmful if they read it before trying, so the interaction has to make reading it a decision rather than an accident.

The family's difficulty spread is now shown too -- "12 Easy, 20 Medium, 8 Hard" -- so a student can see how far the pattern goes beyond the five rungs in front of them.

6d.5 Four more defects, found while enhancing

A rebuild silently destroyed 137 API calls. Descriptions are written once and cached. Rebuilding the index for an unrelated reason overwrote them with nothing, and the only sign was a manifest counter dropping to zero. They are now carried forward across rebuilds, matched on the exact member set rather than on family id -- ids are positional over the sorted community list, so one inserted community rennumbers everything after it and a description would land on the wrong pattern.

The new cache broke test isolation immediately. A global, file-backed cache means a test that exercises query understanding reads whatever a previous *live* run happened to leave on disk. It announced itself at once by breaking a test that monkeypatches the model call -- the answer was already cached, so the patched call never ran. Every test now gets a throwaway cache file.

A test passed while producing nothing. The extracted `derive_families` read the Louvain resolution from config, and at the production value of 8.0 a twelve-problem fixture shatters into communities below the size threshold. The test asserted on graph statistics, which were fine, and never noticed that the family list was empty. Making resolution a parameter fixed the test; asserting on the families themselves fixed the test's blind spot.

A docstring described the previous design. The families module still opened by explaining that acceptance rate is "the only difficulty signal in the corpus finer than Easy/Medium/Hard", several hours after approval had been added beside it. Stale documentation of this kind is worse than none, because it is specific and confident.

6d.6 Making the central promise checkable

The README opens by saying every number in it was measured by a script in this repository. Asked directly whether the project was finished, checking rather than answering turned up that the test count was quoted twice, in two places, and both were wrong -- one by a single test, the other by sixty-eight. Every *measured* figure around them was correct; it was the prose that had drifted.

`scripts/check_docs.py` now compares fourteen manifest figures against the documentation and obtains the test count by running the suite rather than trusting a comment. It caught real drift on its first proper run.

It also produced a small lesson of its own. The first version invoked pytest with `-q`, and `pyproject.toml` already sets `-q` in `addopts` -- so the suite ran at `-qq`, which suppresses the summary line the checker was parsing. It reported "could not determine the test count" while the tests passed perfectly well. A checker that fails silently in the direction of "nothing to report" is worse than no checker, and the fix was to stop passing a flag the project had already set.

6d.7 What the interface showed that the engine did not

The enhancements were verified through the engine's output, which is the easy half. Driving the actual interface showed everything rendering correctly -- descriptions as headlines, curated-link citations, "follows on from" on rungs, related families -- and one thing that only exists at the presentation layer.

The family's difficulty breakdown rendered as "**3 Easy, 6 Hard, 13 Medium**", sorted alphabetically by a `sorted()` on dictionary keys. No number is wrong. It simply places Hard between Easy and Medium, directly beneath a ladder whose entire purpose is to teach that ordering.

Worth noting what did *not* turn out to be a bug: the hint panel did not appear, which looked like a failure until the data said otherwise. Next Greater Element I has no hints, and 71% of the corpus does. The feature correctly showed nothing.

Part 6e - The published numbers were not reproducible

Found by asking a question that had never been asked: does this work from a fresh clone?

Tests passed. Lint passed. The index built. And the manifest came out different: **138 families where the repository documented 137**, largest family 38 where the README said 42, six duplicated names where it said ten.

Narrowing it down

Everything upstream of the clustering was byte-identical between the two checkouts, verified by hash:

artefact	fresh clone vs local
Raw dataset	identical
corpus.json	identical
embeddings.npy	identical
Graph edge set	identical
Graph edge <i>insertion order</i>	identical
Graph node order	identical
Louvain partition	472 communities vs 473

Two hypotheses were wrong before the right one. It was not hash randomisation: forcing PYTHONHASHSEED to 0, 1 and 2 gave byte-identical partitions. It was not edge insertion order feeding a different dict traversal, which was the more plausible guess -- the order hashes matched exactly.

The difference was **numpy 2.5.2 against numpy 2.4.6**, with the same networkx and the same seed.

Why a numeric library changes a graph algorithm

Louvain moves each node to whichever neighbouring community offers the largest modularity gain. On a graph this sparse, a great many of those gains are equal or nearly equal, and the winner is decided by floating-point comparison. Change anything about how those sums are computed and a handful of ties fall the other way; each one shifts a node, and a few shifted nodes split or merge a community.

The seed is real and it works. It fixes the *order* nodes are considered, not the arithmetic used to compare them.

The part that actually mattered

The version difference was not bad luck. The lockfile carries numpy twice, keyed on Python version -- 2.4.6 below 3.12, 2.5.2 at or above -- because `requires-python` was `>=3.11` and

nothing pinned an interpreter.

My virtualenv had been created months of session-time earlier with an explicit `--python 3.12`. A fresh clone running plain `uv sync` resolved 3.11.

So every number published in the README and this report had been measured on an environment a reader could not reproduce from the repository, and nothing said so. The lockfile was doing exactly what it promised and the promise was narrower than it appeared: it pins dependencies *given* an interpreter.

A `.python-version` file fixes it. CI continues to test 3.11 and 3.12 explicitly, which is worth keeping -- the code should work on both, and the tests assert on behaviour rather than on exact family counts, so they pass on both. Only the default is now pinned.

What this changes about the claim

The docstring said the build was deterministic and that two runs produce byte-identical output. Corrected, the claim is:

- The corpus, BM25 index, embeddings and graph are byte-identical across runs, full stop.
- The clustering is reproducible **against a pinned dependency set**, which is a weaker statement and the true one.

That distinction is easy to lose. "Seeded, therefore reproducible" is the intuition, and it is wrong in the specific way that matters here: a seed controls the randomness an algorithm asks for, not the arithmetic underneath it.

6e.1 The fix for reproducibility broke the build

Pinning the interpreter fixed reproducibility and broke CI, and the failure was invisible from the desk it was made at.

CI tests both supported Python versions by passing `--python` to the sync step. The pin is a `.python-version` file, which every *other* `uv` command reads. So after the sync built a 3.11 environment, the lint step's bare `uv run ruff check` consulted the file, decided the project wanted 3.12, disagreed with the environment sitting in front of it, and failed before ruff ran at all.

Three things about this are worth keeping.

It presented as a lint error and was not one. The step was named "Lint" and it failed, which points attention at the code. Ruff 0.16.5 passes on both interpreters -- confirmed by running the ruff binary directly against a 3.11 environment, which was the only way to check, since this machine's application control policy refuses to execute a 3.11 interpreter at all.

It was found by asking a question, not by an alarm. Nobody was watching the badge. Three pushes went out red while the README claimed CI was green, and the claim had been written *because* CI had been checked once, several changes earlier. A verified claim decays into an unverified one silently.

The fix is scope, not value. The override was correct and applied in one place; it belonged at the job level, where it covers every command rather than the one that happened to carry a flag.

6e.2 Steady-state memory is not the number that matters

The project targets a host with a ~1 GB memory ceiling, and resident memory had been measured at ~700 MB with both models loaded. That looked like 300 MB of room.

It was the wrong measurement. A fresh deployment has no cached indexes -- they are derived data and deliberately uncommitted -- so the very first thing it does is build them, and **the build peaked at 976 MB**. Forty-eight megabytes of headroom, before the web server's own footprint, during the one operation that happens before anything is served. It would very likely have been killed, and the failure would have looked like a mysteriously dead container rather than an out-of-memory error anyone could read.

Finding it took sampling RSS every 20 ms during a build rather than reading it at the end. Steady state is what a process settles at; a host kills you for the peak.

The first fix was wrong. The obvious theory was that holding all 2,830 documents, the encoder's internal length-sorted copy of them, and the accumulating output at once was the cost -- so encoding was rewritten to run in chunks into a preallocated array. That moved the peak from 970 MB to 969 MB, and was removed.

Instrumenting each stage separately located it precisely: resting memory after the encoder loads is 487 MB, and encoding spikes to 976 MB. The cost is the activations of a single forward pass, so the lever is the batch size, not the chunk size.

batch	peak	encode time
64	951 MB	110 s
32	799 MB	110 s
16	687 MB	110 s
8	686 MB	101 s

Encoding took the same time at every setting, so the 264 MB between 64 and 16 is free, and below 16 there is nothing further to win. A full cold build now peaks at **712 MB with 312 MB of headroom**, and finishes faster than before.

One thing worth recording because it did not bite and could have. Changing the batch size changes the padding within each batch, which changes the arithmetic: 1.01% of embedding values moved, by up to 7.5e-08. Given that a numpy version change was already shown to re-partition the graph (6e), that was a live risk. The clustering absorbed it -- the family partition is byte-identical -- but that is a fact about this corpus at this threshold, not a guarantee. The batch size is pinned in config for the same reason the interpreter is.

6e.3 Simulating the deployment before making it

Before deploying, the whole path was rehearsed in a container built to resemble the target host rather than to be convenient:

- it clones the **public repository**, so only committed files exist;
- it installs from `uv.lock`, which is the dependency file Community Cloud

reads first;

- it pins Python 3.12, matching `.python-version`;
- it does **not** prebuild the index, so the application must bootstrap itself;
- and it runs under a hard 1 GB cap with swap disabled.

It worked. The first query triggered the build, memory rose and plateaued at **691 MiB of 1 GiB -- 67%** -- and the app returned ranked results, a named family, and a ladder with its curated-link provenance. No OOM kill.

Two things this caught that reading could not.

A half-finished build would have been permanent. The interface bootstraps when loading raises `FileNotFoundError`. But an interrupted build -- a network timeout, a container restarted mid-encode -- leaves some artefacts written and others not, which loading rejects with a `ValueError` instead. Catching only the first would strand a deployment in a state it could have repaired by itself, on a service where nobody can log in to rerun a script.

Two deployment settings were only ever stated in conversation. The Python version has to be pinned to 3.12 in the host's own settings, or the 3.11 dependency resolution produces 138 families rather than the documented 137. And a Groq key must sit at the *root* of the secrets TOML, because only root-level secrets are exposed as environment variables and this application reads `os.environ` -- nested under a section it is silently ignored and the app quietly falls back to the offline parser. Neither was written down where a deployer would look.

6e.4 A lockfile does not pin the operating system

The pinned interpreter and the lockfile between them fix every version this project depends on, and Part 6e treats that as the end of the reproducibility problem. It is not. Late in the work the development machine began refusing to run the encoder at all:

```
ImportError: DLL load failed while importing _rank_filter_1d:  
An Application Control policy has blocked this file.
```

Nothing in the project had changed. No dependency had moved -- the lockfile was unchanged and the resolved versions were identical. The file the loader was complaining about was present on disk, 128,000 bytes of it. Windows 11's Smart App Control had moved from evaluation mode to enforcement, and it declines to load unsigned native extension modules. SciPy, reached through sentence-transformers by way of scikit-learn, is a large collection of exactly those. The error names a different `.pyd` on each run as the policy works through them one at a time, which makes it read like a corrupted install rather than a policy decision; `import scipy` succeeds while `import scipy.ndimage` fails, which makes it read like a partial one.

Two things about the failure are worth recording. The first is that the test suite did not notice. The whole suite continued to pass, because it substitutes a stub encoder and never reach SciPy at all -- the property that lets them run offline in under twenty seconds is the same property that blinded them here. That is the honest cost of the design, and it is the right trade: a suite that loaded real models would be slow, would need network access, and would still have been useless for catching a policy that had not been enforced when it was written. But it does mean the suite passing is not evidence that the system can encode a query on this host, and those two claims had been quietly treated as one.

The second is that continuous integration did not notice either, for a different reason: it runs on Linux, where the policy does not exist. So did the container. So does the deployment. Every automated check the project has agreed the system was fine, and every one of them was right about the environment it was testing. The failure lived entirely in the gap between them -- a developer's own workstation, the one environment nothing is configured to verify.

The remedy is documented in the README rather than attempted in code, because there is no code that can help. Smart App Control cannot be re-enabled once disabled without reinstalling Windows, so advising a reader to turn it off is advising a one-way change to their machine's security posture in order to run a study tool. Running under WSL2 or in the container sidesteps the policy without weakening the host, and both were already supported. What this cost was not a fix but a correction to a claim: the project reproduces exactly given the pinned interpreter and the lockfile *and a host that will load the binaries they name*, and only the first two of those three were previously written down.

6f Derived data was excluded on principle, and the principle was wrong

Build outputs do not belong in version control. It is one of the few rules in this trade with almost no exceptions: they are large, they are binary, they change on every build, they bloat history permanently, and they can be regenerated from the inputs that *are* tracked. The index here is textbook derived data -- a 15 MB embedding matrix, a BM25 structure, and two JSON files, all reproducible from the corpus and the lockfile. It was gitignored without much thought, because the rule is a good one.

The deployment is what showed the rule had an edge, and the edge was this project. Streamlit Community Cloud gives a container that is stopped when idle and started again on the next visit. With no index in the repository, the application did the thing it was carefully designed to do: it noticed the artefacts were missing and built them. That takes about three minutes of sustained CPU -- fetching the corpus, then encoding 2,830 documents through a transformer. On a paid host that is a one-time cost. On a free tier that meters CPU to share it between thousands of applications, it is a bill payable on every cold start, and the platform eventually declined to keep paying it:

Your app has been throttled... we've temporarily reduced its CPU to keep the platform healthy for everyone.

What makes this worth recording is that no individual decision was wrong. The bootstrap-on-missing-index behaviour is correct, and Part 6e.3 argues for it at some length: without it a fresh host fails at startup with a message about running a script nobody can run. Excluding derived data is correct. The two correct decisions compose into a system that rebuilds a transformer index every time a student stops reading for an hour.

The fix was to commit the index -- 15 MB, one time, into a repository that is now 6.5 MB packed after compression. A fresh clone loads it in **0.77s** with no build at all. The bootstrap path stays exactly as it was, because it is still right for the case it was written for; it simply stops being the common case. Nothing was removed, and the change is four lines in `.gitignore`.

Three smaller things fell out of it, all of which would have been invisible until they caused damage.

The first is that `.gitignore` cannot re-include a file beneath an excluded directory. Writing `artifacts/` and then `!artifacts/index/` does nothing: git does not descend into an excluded

directory, so no negation inside one can ever match. The exclusion has to be written `artifacts/*` so that git still walks the directory and can see the exception. This fails silently -- `git add` reports nothing, and the index simply is not there.

The second is that committing a binary means trusting git's binary detection, which is a heuristic over the first few kilobytes of the file. On Windows, a file misclassified as text has its line endings rewritten on checkout, which for an embedding matrix means silently corrupted floats and subtly wrong neighbours forever, with no error at any point. A fresh clone was checked byte-for-byte against the original and all six artefacts matched, so the heuristic was in fact correct -- but a heuristic that happens to be right is not the same as a guarantee, and `.gitattributes` now states the file types outright rather than leaving it to be inferred.

The third is that `make clean` still read `rm -rf artifacts`, under a comment promising that "everything here is rebuildable". That was true when it was written and stopped being true the moment the index was committed, and nothing connected the two. Running it would have deleted fifteen megabytes of tracked files and left a working tree full of deletions -- recoverable, but only by someone who understood what had happened. Worse, a cleaned checkout deployed as it stands goes straight back to rebuilding the index on every cold start, which is the throttle this whole section is about. It now uses `git clean`, which will not remove tracked files whatever the ignore rules say.

That one is the most instructive of the three, because it is not really about git at all. Committing the index changed what the word "derived" meant in this repository, and every place that had quietly encoded the old meaning -- a gitignore rule, a binary heuristic, a clean target, a sentence in the README -- was wrong from that moment, silently, until someone went looking. A decision does not finish where it is made.

The general lesson is narrower than "commit your build outputs", which remains bad advice. It is that a rule about *storage* was applied to a situation whose real constraint was *compute*, and the two only look alike until something meters one of them.

6g The metric scored one string and the interface displayed another

Every family has two labels. `name` is derived deterministically from the tags its members share -- "Subarray / Sliding Window / Prefix Sum" -- and `headline` is the model-written description where one exists, falling back to the name where it does not. The description is the better line almost always, which is why the interface leads with it and demotes the name to a caption.

`family@1` scores the name and the tags. The interface displays the headline. Those are different strings, and nothing measured the second one.

The consequence was found by running the query this project is named for:

```
Q: I keep failing problems where you shrink a window from the left
family : Subarray / Sliding Window / Prefix Sum      <- scored, hit
headline: use prefix sums to compute subarray sums quickly  <- displayed
```

The retrieved problems were right -- *Sliding Window Maximum, Minimum Size Subarray Sum* -- the family was right, the ladder was right, and `family@1` recorded a hit, correctly, about the string it was checking. The student was told the pattern was prefix sums. Naming the pattern is the one thing this system exists to do, and on its own headline example it named the wrong one

while every metric read green.

The cause is that the family genuinely holds two techniques. It is tagged both Sliding Window and Prefix Sum, and it contains *Minimum Size Subarray Sum* alongside *Range Sum Query 2D*. A description is written once per family with no knowledge of any query, so for a mixed family it must commit to one of the techniques, and it will be wrong for everyone who asked about the other.

The first attempt at measuring the gap was itself wrong, and the way it was wrong is worth keeping. Scoring the headline by substring gave 0.583 against 0.833 for the name, which looks like a devastating 25-point overstatement. Reading the three disagreements showed that two of them were the metric failing rather than the system:

query	name (scored)	headline (displayed)
shrink a window from the left	Subarray / Sliding Window / Prefix Sum	"use prefix sums ..."
reverse a linked list	Linked / Linked List / Recursion	"reordering nodes by pointer manipulation"
coin change, fewest coins	Greedy / Array	" DP to find minimum number of items"

Only the first is a defect. The second paraphrases rather than repeats, and is better than the name it replaced. The third is better still: the tag-derived name says *Greedy*, which is wrong for coin change, and the description says DP, which is right -- the substring test scored the good line as a miss and the bad line as a hit. Publishing 0.583 would have been publishing an artefact of literal string matching, so it is not published, and the number appears here only as a record of an instrument that did not work.

What distinguishes the real defect is that the description names a *sibling tag of the same family* while omitting the one asked about. That is the signature of having chosen one technique out of several, as opposed to describing the only one there is in different words. Both the fix and the check now use exactly that condition.

The fix reorders the two labels rather than changing either. When the family covers the technique that was asked about and the description instead names a sibling tag, the name leads and the description moves to the caption. Nothing is hidden and no other query is affected; `hit@5`, `family@1`, `ladder_hit` and the latencies are all unchanged. The evaluation now reports `headline_names_wrong_technique`, scored against the headline the interface would actually render, which reads 0 with the fix in place and 1 without it.

The first version of the fix was applied to the ladder title alone, which left the same wrong label in `explain()` -- the sentence attached to every individual result, and the thing the brief calls the reason per recommendation. That is the worse half to miss: the title appears once per answer and the reason appears once per result, so the wrong technique was being asserted five times for every one time it was corrected. `explain()` had the family but not the query, so the technique is now threaded through the pipeline into it. Half-fixing a display defect is easy precisely because the half that was fixed is the half you were looking at.

It is deliberately a count rather than a rate. A rate invites being quoted as a quality score, and this is a tripwire for one specific defect on twelve scored queries. The more general problem it points at -- that a single line cannot describe a community holding two techniques -- is not solved by reordering two labels, and is recorded in the limitations instead.

6h Thoroughness that produced a worse artefact

The Dockerfile built the index inside the image. That was deliberate and, at the time, right: the image should carry everything it needs, so a container starts without downloading a corpus or encoding three thousand documents while someone waits. Baking the artefacts in is the whole reason the image is justified at 4.25 GB.

Committing the index made it wrong, and in a way that looked like diligence. A build inside the image has no API key -- the key lives in a gitignored `.env` and is never in a build context -- so `build_index.py` running there produces 137 families with **zero** model-written descriptions, where the committed index has 120. Three minutes of build time were being spent manufacturing a strictly worse copy of an artefact already sitting in the repository, and then shipping it.

The effect is visible in the product. Asked about shrinking a window, the old image answered:

```
in family 'Heap (Priority Queue) / Array' (31 problems)
```

where the deployed application says:

```
in family 'DP over array with previous element constraint' (31 problems)
```

Both name the same 31-problem community. One says which shelf it sits on; the other says what the problems have in common, which is the entire point of generating descriptions at all. Every family in the image had silently fallen back to its tag-derived name.

Both manifests recorded this the whole time -- `described_by_model: 0` against `described_by_model: 120` -- in a field written specifically so that this property could be checked. Nothing checked it. The manifest was being produced diligently and read by a script that verifies the *documentation* against it, which is a narrower thing than it sounds: `check_docs.py` compares the numbers in the README to the numbers in the manifest on the machine that built it, and has no opinion about a second manifest inside a container image.

The fix is to copy the committed index instead of rebuilding it, which is the same decision as Part 6f applied one layer out, and it is strictly better on every axis: the image becomes byte-identical to the deployment (verified by hashing `embeddings.npy` on both sides), it gains 120 descriptions, and the build step falls from 192 seconds to 53. The index is also loaded during the build rather than only at runtime, so a bad or missing copy fails the build instead of the first query a user makes.

Two details were needed to make it work. `.dockerignore` had to gain `!artifacts/index/`, and unlike the equivalent problem in git this negation does take effect: the Docker builder walks the entire context and applies its patterns in order, rather than declining to descend into an excluded directory. And the build context stays small, because the exception admits 15 MB of index while continuing to exclude the virtualenv and the Hugging Face cache that made the original context 1.6 GB.

The uncomfortable part is how long this survived. The image had been built, run, and measured; a query had been put through it and answered correctly. It was never compared against the thing it was supposed to be a copy of.

Asking a second question of the same image -- why it took so long to build -- found the other half. The image was 4.25 GB and the obvious answer is PyTorch, which is true and incomplete. Measuring the layers put 2.86 GB in the dependency install, while the virtualenv that install produces is only 1.33 GB. The missing 1.5 GB was uv's download cache, sitting in the image beside the virtualenv it had just been copied into:

/root/.cache/uv	1367 MB
/app/.venv	1365 MB

Every dependency was present twice. The cause is `UV_LINK_MODE=copy`, which is set two lines above and is correct: it makes uv copy wheels out of the cache rather than hardlink them, which is what a container wants when the two may live on different filesystems. What nobody then asked is what happens to the cache afterwards. Nothing does. It is written into the layer, compressed on every export, pulled by everyone who pulls the image, and read by nothing.

`--no-cache` removes 1.81 GB and takes the image to 2.44 GB, with the same artefacts, the same results and the same 862 MiB peak. Docker's layer cache is untouched, because that is a different cache from uv's -- a distinction the original line quietly conflated.

What remains is genuinely irreducible: 695 MB of PyTorch, 184 MB of model weights, 152 MB of pyarrow arriving through Streamlit, 15 MB of index, and about 2 MB of this project. A CPU transformer stack has a floor and this is close to it. The build spends roughly a minute installing that, fifty seconds downloading and warming the two checkpoints, and the rest exporting layers -- which is precisely why 1.81 GB of dead weight was worth finding.

A third defect surfaced only from watching a build that should have been instant take 150 seconds. `pyproject.toml` sets `readme = "README.md"`, because hatchling needs it to build the project wheel, and the Dockerfile duly copied it in the same instruction as the dependency manifest:

```
COPY pyproject.toml uv.lock README.md ./
RUN uv sync --frozen --no-dev --no-install-project --no-cache
```

Docker invalidates a layer when any of its inputs change, and everything below it with it. So editing a sentence of prose invalidated the dependency install, the project install, and the model warm-up: a documentation typo cost a hundred seconds. The README was in that instruction because the wheel build needs it, which is true, but the wheel is built by the *second* sync, after the source is copied. Moving it there takes a README-only rebuild from 150 seconds to 70.

The general shape is the same as the cache duplication above. Each line was individually defensible -- the readme is a real dependency of the wheel, and copying dependencies before code is the standard advice. What went unasked is which of them change together. Layer ordering is a claim about rate of change, and putting the most frequently edited file in the repository directly above the most expensive step is the exact inversion of the intended one.

6i A check that reported failure and returned success

The PDF renderer rasterises every page and inspects it for the signature of a glyph failure. It is the right check, and it is how the stranded-rule defect in Part 6e was caught at all: the page was blank, the check said so, and a person read the line. Then it returned zero.

```
wrote docs/Pattern-Ladder-report.pdf (45 pages, ...)
WARNING blank pages: [43]
$ echo $?
0
```

A check that reports a failure through stdout while telling its caller everything is fine cannot gate anything. It cannot fail a build, it cannot stop a release script, and the only reason it ever worked was that someone happened to be watching. The verification was real; the plumbing made it advisory.

The same review found the renderer's no-argument form rendering one document when two are committed. `make report` refreshed the report and silently left the corrected brief at whatever it had been, and printed a success for the file it did render. It is the same shape as the output-path defect in Part 6e: a command that regenerates most of what it claims to, and says nothing about the rest.

Both are one-line fixes -- a non-zero return, and a default of every document rather than one. What is worth keeping is that neither was a bug in any check. Every check here was correct and had been correct all along. What was wrong was the assumption that a correct check, having been written and having produced the right answer, was therefore doing something. Between a check and a consequence there is always a wire, and nothing about writing the check guarantees the wire.

6j A committed artefact that changed every time it was built

The PDFs are committed, and the renderer produced different bytes on every run from identical source. `reportlab` stamps the wall-clock time into `/CreationDate` and `/ModDate` and generates a random document `/ID` unless told otherwise, so two renders of an unchanged document are two different files.

The cost is not aesthetic. It made a question unanswerable: *was this PDF built from this markdown?* A diff always showed the binary as modified, so the signal that would have answered it was permanently saturated. Editing the report and forgetting to re-render is invisible under those conditions -- the source is right, the committed PDF is wrong, and the only symptom is a reader receiving a document that disagrees with the repository. Nothing in the project could have detected it.

`rl_config.invariant = 1` fixes the timestamp and derives the ID from the content, after which an unchanged document renders to identical bytes. That one line is not the interesting part; what it enables is. Determinism converts a question that required reading forty-nine pages into a byte comparison, so `check_docs.py` now re-renders every document and compares it to the committed PDF.

It found a stale one immediately -- the corrected brief, left behind by the very change that made the check possible. That is a fair illustration of the whole difficulty: the staleness had presumably existed at various points for some time, and became visible only at the moment something was capable of looking.

Part 7 — What the research changed

7.1 The reranker finding is not unique to this corpus

The measured result — cross-encoder reranking lowering recall while raising precision — matched published work closely. One study reports total recall falling from 0.828 to 0.733 under cross-encoder reranking, describing it as an expected consequence of truncating to a fixed depth, and concludes that fusing both ranked lists through RRF yields higher recall *and* precision than either alone.

That is exactly the architecture adopted here, and finding it independently after measuring it is the strongest form of confirmation available without a larger evaluation set. It also raised confidence that the 20-query result was signal rather than noise.

7.2 BM25 parameters, checked and left alone

Practitioner guidance suggests `k1` in [1.2, 2.0] and `b` in [0.1, 1.0] are worth tuning, particularly for corpora with variable document lengths — which this is, since a two-line statement sits next to a page of worked examples.

Swept across 64 configurations. `k1` changed pool recall by at most 0.007; `b` by at most 0.010; **hit@5 did not move at all**. The `bm25s` defaults (`k1=1.5`, `b=0.75` — note 1.5, not the commonly quoted 1.2) are kept.

This negative result is more useful than it looks. Pool recall is **1.000**: the correct answer reaches the candidate pool on every single query. So every remaining error is in *ranking*, not retrieval, and effort spent on retrieval tuning is effort wasted. That single number redirected the rest of the work toward the ladder.

7.3 Candidate encoders, evaluated on this corpus rather than on MTEB

Published MTEB averages are a poor basis for this decision — they average dozens of tasks over prose that looks nothing like a problem statement, and the constraint here is a CPU-only deployment under a memory ceiling. Four candidates were benchmarked on this corpus and this evaluation set: the incumbent `all-MiniLM-L6-v2`, `bge-small-en-v1.5`, `granite-embedding-small-english-r2`, and the static `potion-retrieval-32M`.

model	params	corpus encode	query	dense-only hit@5	hybrid hit@5
all-MiniLM-L6-v2	22.7M	98 s	7.3 ms	0.900	0.900
bge-small-en-v1.5	33M	175 s	12.1 ms	1.000	0.950
granite-embedding-small-english-r2	47M	—	—	—	—
potion-retrieval-32M	32M	—	—	—	—

The granite model downloaded and began encoding but had not finished after forty minutes -- against 98 and 175 seconds for the others -- and was abandoned rather than reported with an invented figure. The static model needs an optional package that is not a dependency, so it was skipped rather than guessed at. **Neither was measured, and neither is claimed.**

bge-small-en-v1.5 is clearly the better *encoder*: it puts the expected problem in the dense top-5 on every query, where MiniLM manages 18 of 20. The extra cost is trivial at query time -- 12 ms against 7 ms, next to a 1,200 ms reranker.

And swapping it in made the system worse. Section 7.4 is about why.

7.4 A better encoder that made the system worse

With bge-small-en-v1.5 substituted and everything rebuilt:

	MiniLM	bge-small
hit@5	0.95	0.90
Ladder contains the canonical problem	0.85	0.75
Ladder from the expected family	0.833	0.583
Mean rank of hits	1.53	1.33
Corpus coverage by families	86.3%	98.3%
Inferred graph edges	2,463	3,543

Precision improved and everything else fell over.

The cause is that the embeddings are not only used for ranking. The graph backfill thresholds on **raw cosine similarity**, and different models produce different similarity *distributions*. bge-small's similarities sit higher, so the same 0.65 floor admitted 3,543 inferred edges instead of 2,463, pushed family coverage to 98.3%, and redrew the family boundaries that the ladder depends on.

Re-sweeping the graph parameters specifically for bge-small (scripts/sweep_family.py) recovered ladder-family accuracy only to 0.75, still below MiniLM's 0.833. So the decision is to keep MiniLM -- not because it is the better model, but because it is the better model *for this system as tuned*.

The lesson, and it generalises well beyond this project: a component cannot be evaluated in isolation when something downstream consumes more than its output ranking. Anything that thresholds on a model's raw scores is coupled to that model's score distribution, and swapping the model silently changes the meaning of every threshold. The benchmark that said "this encoder is better" was correct and still led to the wrong decision.

Part 8 — Operating findings and measured numbers

Every number below was produced by a script in this repository. Nothing is estimated.

Corpus

Upstream rows	3,549
Paid-only excluded	716
Empty statements excluded	3
Retrievable	2,830
Easy / Medium / Hard	752 / 1,417 / 661

Graph and families

	curated only	final
Edges	1,932	4,395 (1,932 curated + 2,463 inferred)
Isolated problems	1,074 (38.0%)	298 (10.5%)
Components	1,248	341
Families (≥ 5 members)	60	137
Coverage	49.2%	86.3%
Largest family	63	42
Family size distribution	—	16 tiny, 70 small, 50 medium, 1 large
Name coherence	0.790	0.845
Tag agreement (NMI)	0.618	0.667
Families sharing a name with another	-	10
Families with a model-written description	-	120 of 137
Families with a recorded next-pattern link	-	111 of 137

Retrieval

Both columns are real runs over the same twenty queries. The only difference is whether query understanding used the offline parser or one Groq call.

metric	offline parser	live model
hit@5	0.95 (19/20)	1.00 (20/20)
hit@5, oblique phrasing	1.00 (6/6)	1.00 (6/6)
hit@5, literal phrasing	0.929 (13/14)	1.00 (14/14)
Pool recall (top 50)	1.000	1.000
Ladder contains the canonical problem	0.85	0.85
Ladder from the expected family	0.833	0.917
Mean rank of hits	1.53	1.80
Mean approval of recommended problems	0.910	0.909
Queries parsed by the model	0	20/20

The live API

Measured over the same twenty queries against `openai/gpt-oss-20b` on the free tier.

Calls succeeded	20/20
Prompt tokens per query	272
Completion tokens per query	100
Total per query	372 (max 474)
Model latency	0.91 s mean, 1.48 s p95
Throughput ceiling	21.5 queries/min
Daily ceiling	538 queries/day
Cost if metered	\$0.05 per 1,000 queries

Two things in that table are worth stating plainly. The binding constraint per minute is tokens, not requests (8,000 TPM against 30 RPM), and the binding constraint per *day* is also tokens -- 200,000 TPD divides to 538 queries, below the 1,000 request-per-day cap. Anyone budgeting against the request limits would over-estimate capacity by nearly a factor of two.

Latency, measured on a 12-core desktop CPU

stage	time
Query understanding, offline parser	<1 ms
BM25 + dense + fusion	~15 ms
Cross-encoder over 50 candidates	~1.2 s
Family lookup and ladder	<1 ms
End-to-end, warm, median	1.2 - 1.6 s
End-to-end, p95	1.6 - 1.8 s
Load every cached index	0.06 s
Cold start to first answer	20.8 s

Warm latency is a range rather than a point, and honestly so: repeated runs of the same twenty queries on the same machine produced medians between 1.23 s and 1.58 s depending on what else was competing for CPU. The reranker is around 90% of that and is CPU-bound, so quoting a single figure would claim a precision the measurement does not have. The truncation comparison below was taken in one sitting, so its *relative* numbers are sound even though the absolute ones drift.

The reranker dominates warm latency. Truncating candidates to 600 characters cut it from 2.85 s to 1.32 s while returning the identical top result on every query tested. Sub-second is reachable at 300 characters, but that is where the top result starts to move.

Cold start is a different story, and the original plan's "under two seconds" claim accounts for the wrong cost. Loading every cached artefact -- corpus, BM25 index, embedding matrix, families -- takes **60 milliseconds**. The 20 seconds is reading and initialising ~90 MB of model weights, which no amount of index caching addresses. The Dockerfile bakes the weights into the image, so it is paid once per container rather than per request.

Memory

stage	resident set
Bare Python	19 MB
After importing torch and sentence-transformers	73 MB
After loading every cached index	93 MB
After both models are resident	~700 MB
Peak during a cold index build	712 MB

The indexes cost 20 MB; PyTorch and the models cost 600. Against the ~1 GB ceiling of the free hosting tier this fits with about 320 MB spare, before Streamlit's own process is counted. Reproduce with `scripts/measure_memory.py`.

Engineering

Tests	236, all offline, no secrets
Lint	ruff clean
Python versions in CI	3.11, 3.12 (green on every push)
Pinned interpreter	3.12, via .python-version
Parameter sweeps	4 scripts, ~230 configurations
Offline parser technique recall	22 of 24 probe queries

Part 9 — Running and extending it

```
uv sync --extra dev
uv run python scripts/build_index.py          # ~2 minutes
uv run streamlit run src/pattern_ladder/app.py
```

No API key needed. The offline parser runs the whole system and the interface labels the parse as rule-based.

```
uv run pytest -q                               # 147 offline tests
uv run ruff check .
uv run python scripts/evaluate.py --k 5 --verbose
uv run python scripts/render_pdf.py docs/report.md
```

To change a tuned constant, edit `config.py` — but re-run the sweep that chose it, and update the comment. A constant without a measurement behind it is how this project acquired several of the bugs in Part 6.

To add evaluation queries, extend `eval/smoke_queries.json`. Give `expect_family_contains` only where a single family is unambiguously correct; leaving it null is better than inventing an answer.

To swap the encoder, change `DENSE_MODEL` and rebuild. Run `scripts/compare_encoders.py` first — it measures on this corpus rather than on a leaderboard.

To add a retriever, produce a `list[(doc_id, score)]` and add it to the list passed to `reciprocal_rank_fusion`. Nothing else needs to know it exists; RRF does not care where a ranking came from, which is precisely why the reranker could be folded in as a third ranker.

Part 10 — Limitations

The live model path is verified, but on twenty queries. It now runs, and runs well (20/20 calls, hit@5 of 1.00), but that is the same small evaluation set with the same caveats. Nothing here establishes how it behaves on queries unlike these, and the first real call found three defects that months of offline testing had not — so the honest reading is that this path has been exercised, not that it has been proven.

Deployed, and throttled within a day. It runs at <<https://pattern-ladder-1.streamlit.app/>>. Everything a deployment needs was checked rather than assumed: Community Cloud reads `uv.lock` ahead of every other dependency format, the lockfile resolves CPU-only PyTorch on Linux with no NVIDIA packages at all, and the app builds its own indexes on first load (112 seconds measured) rather than failing on a fresh host that has none. It then ran, returned correct results, and was CPU-throttled by the host -- for that last property. Bootstrapping on demand is the right behaviour for a missing index and the wrong default for a free tier that meters CPU, because every cold start paid for it again. See Part 6f.

Nothing compares the image's artefacts to the repository's automatically. They are identical today, and that was checked by hand rather than by a test: comparing them properly means building a 4.25 GB image, which no offline suite can do.

A family can hold two techniques, and one line cannot describe both. The community tagged both Sliding Window and Prefix Sum contains *Minimum Size Subarray Sum* and *Range Sum Query 2D*, and its description -- written once per family, with no knowledge of any query -- necessarily commits to one of them. Part 6g stops the wrong one from leading the display, which is a presentation fix, not a clustering one. Splitting such communities would be the real answer and was not attempted: the resolution parameter was already swept against `family@1`, and raising it further fragments families that are not mixed.

The evaluation set is 20 queries, written by one person who knew the system. It is a tripwire against regressions, not a benchmark. `family@1` is scored on 12 of them, so one query moves it 8 points. No result here should be quoted as a general claim about retrieval quality.

No human relevance labels exist. There is no measurement of whether ladders are *pedagogically* good — only whether they contain the problem a reasonable person would expect. Whether this actually helps a student learn faster is unmeasured, and the honest answer is that nobody knows.

The system cannot say "nothing matched." Dense retrieval is exhaustive and returns its nearest rows however distant, so a meaningless query gets confident-looking results. Pinned by a test rather than hidden.

Ten of the 137 tag-derived names are not unique. Four separate families are all called "Tree / Binary Tree". Building with descriptions enabled gives 120 of them a distinguishing phrase, but 17 keep a name that may be ambiguous, and the underlying naming function still has no uniqueness guarantee.

The model does not always read a query the same way. Nineteen of twenty evaluation queries were bit-identical across three repeats at temperature 0; one was not. Temperature 0 is greedy decoding, not a reproducibility guarantee. A single live evaluation run can therefore differ by one query, and one observed run scored 0.95 rather than 1.00.

The image is 2.44 GB, of which 695 MB is PyTorch. It has been rebuilt against the current commit and run under a hard 1 GB cap, answering a real query at a peak RSS of 862 MiB with artefacts byte-identical to the committed index. See Part 6h for the two defects rebuilding it revealed.

14% of the corpus is in no family and returns results with no ladder.

A cross-encoder trained on web search is being used on problem statements.

`ms-marco-MiniLM` was trained on Bing queries and web passages. That domain gap is a plausible explanation for why it hurt recall here, and it was not investigated further.

Part 11 — Glossary

Acceptance rate — the share of submissions to a problem that pass. Used here as the only difficulty signal finer than Easy/Medium/Hard.

BM25 — the standard lexical ranking function. Scores documents by query-word occurrence, weighting rare words more, saturating repetition, and normalising for length.

Bi-encoder — an arrangement where query and document are embedded separately and compared. Fast, because documents are embedded once offline.

Community detection — finding groups in a graph that are more connected internally than externally.

Corpus — the fixed collection of documents being searched.

Cosine similarity — the angle between two vectors, used as a measure of semantic similarity. Equals the dot product when both are length 1.

Cross-encoder — a model that reads a query and a document together and scores the pair. Accurate and slow; used to rerank a shortlist.

Dense retrieval — retrieval by embedding similarity rather than word overlap.

Embedding — a list of numbers representing a text's meaning as a position in space.

hit@k — the share of evaluation queries whose known-correct answer appears in the top k results.

Hybrid retrieval — running lexical and dense retrieval together and merging.

Inverse document frequency (IDF) — weighting that makes rare words count for more.

Isolated node — a graph node with no edges.

Louvain — the standard community-detection algorithm. Stochastic, so it needs a seed; has a resolution parameter controlling community size.

Modularity — how much more internally connected a grouping is than chance would predict. Trivially high on a fragmented graph.

NMI (Normalised Mutual Information) — agreement between two groupings of the same items, from 0 (unrelated) to 1 (identical).

Normalisation (of a vector) — scaling to length 1, so cosine similarity becomes a dot product.

Precision — of what you returned, how much was correct.

Rank fusion — merging several ranked lists into one.

Recall — of what you should have returned, how much you did.

Reranking — reordering a retrieved shortlist with a more expensive, more accurate model.

Resolution (Louvain) — the parameter controlling community granularity. Higher gives more, smaller communities.

RRF (Reciprocal Rank Fusion) — rank fusion by summing $1/(k + \text{rank})$ across retrievers, ignoring scores entirely.

Stemming — reducing words to a common root so "shrinking" matches "shrink".

Stopwords — extremely common words ("the", "of") carrying little retrieval signal.

Tokenisation — splitting text into the units that get indexed.